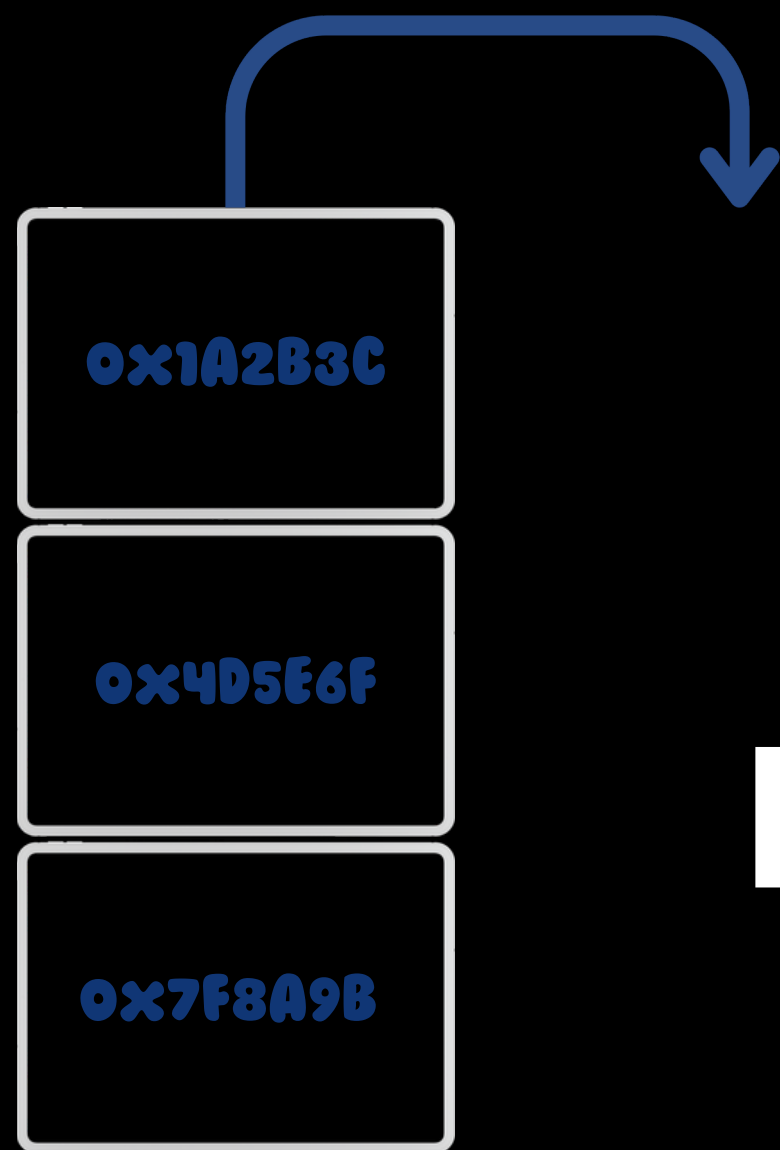
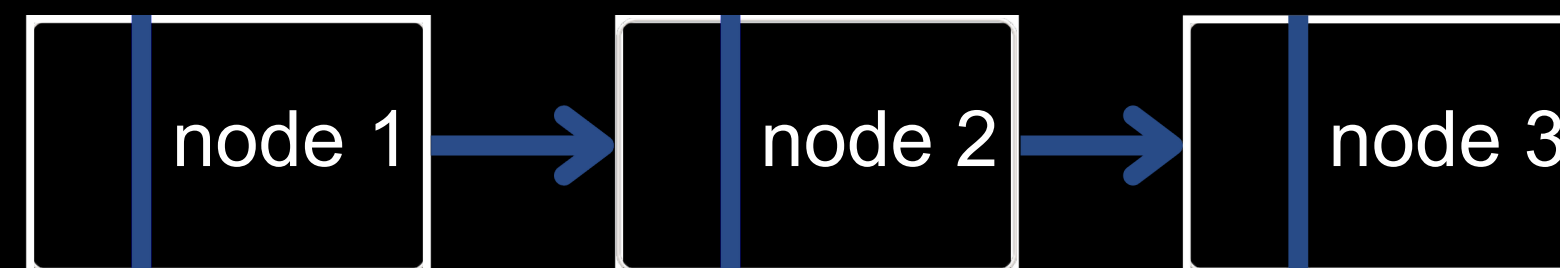
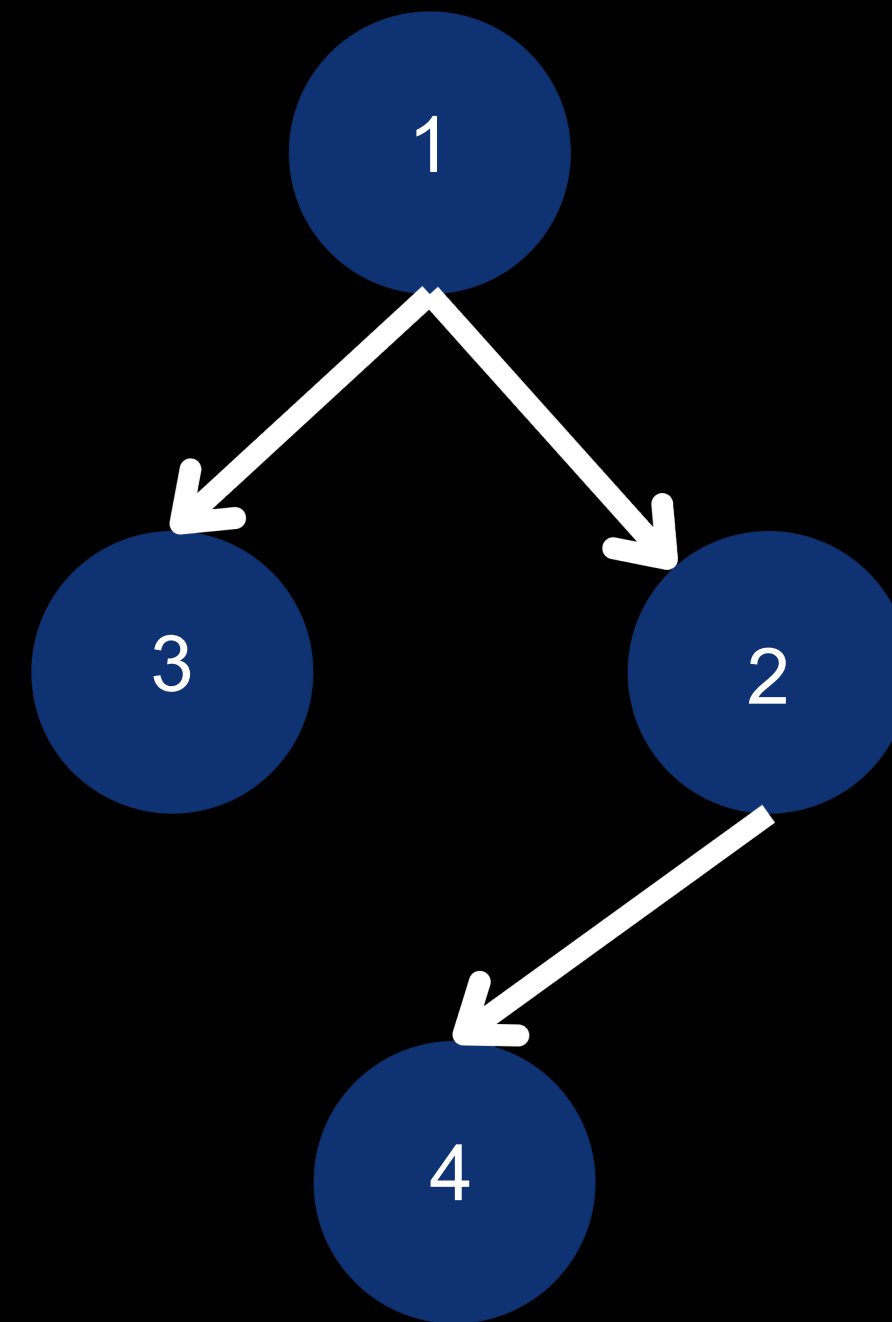




DATA STRUCTURE



essammohamedst1@gmail.com

BFS

BFS algorithm

standard BFS implementation puts each vertex of the graph into one of two categories

- Visited
- Not Visited
- The purpose of the algorithm is to mark each vertex as visited while avoiding cycles

How BST algorithm works

Start at the Root: Visit the root node (Level 0)

Move to Children: Visit all children of the root from left to right (Level 1)

Move Deeper: Visit all "grandchildren" (Level 2), and so on

The Mechanism of BFS Using a Queue

Because BFS explores horizontally, it cannot easily use recursion (which naturally goes deep)
Instead, it uses a Queue data structure (First-In, First-Out)

The Algorithm

- Create an empty Queue

- Enqueue the Root node

- While the queue is not empty:

 - Dequeue a node and "visit" it (e.g., print its value).

 - Enqueue the node's Left child (if it exists).

 - Enqueue the node's Right child (if it exists)

DFS

Depth-First Search (DFS) is the opposite of BFS

Instead of exploring level-by-level, it dives as deep as possible down one branch before backtracking to explore other paths

In a Binary Search Tree (BST), DFS is the most common way to interact with the data because it respects the hierarchical relationship of the nodes

Remember that we can make DFS traversal in 3 ways:

- Inorder
- Preorder
- Postorder

How DFS Works

Start at the root

Follow a path (usually the left child) until you hit a leaf node (a node with no children)

"Backtrack" to the nearest ancestor that has an unexplored child

Repeat until all nodes are visited

Type	Order	Result in a DFS
Inorder	Left -> Root -> Right	Returns values in sorted ascending order.
Preorder	Root -> Left -> Right	Useful for creating a copy of the tree.
Postorder	Left -> Right -> Root	Useful for deleting a tree (children before parents).

BST Successor

Inorder Successor

what is the inorder successor : is the **directly next** largest node of the given node

explaining

in a line of people sorted by height, the successor of a specific person is the one **standing directly to their right**.

While everyone in that right-hand line is taller than the person, the successor is the **shortest among them**

This makes the successor taller than the person, but shorter than everyone else further down the line

in another words

the successor of a node is the node with the **smallest** key **greater than the given node's key**

How to Find the inorder Successor

There are two possible cases

Case 1

If the node has a right subtree, the successor is the leftmost node in the right subtree

The Logic: If there are people to your right, the "immediately taller" person must be among them

In a BST, the right subtree contains all larger values

The leftmost node in that subtree is the smallest of those larger values

Visual: Go Right (larger), then Left (smallest of the larger)

Case 2

If the node has no right subtree, we traverse up using the parent reference until we find a node where the given node is in the left subtree

The Logic: If there is **no one in your right subtree**, you have already visited everyone **"under"** you

The person "immediately taller" than you must be an ancestor who hasn't been fully visited yet
You **climb up** the tree

As long as you are coming from a right child, you are still **"taller"** than the parent (**so the parent can't be the successor**)

The moment you find a parent where you are the left child, you've found the person who is "just barely" **taller than your entire branch**

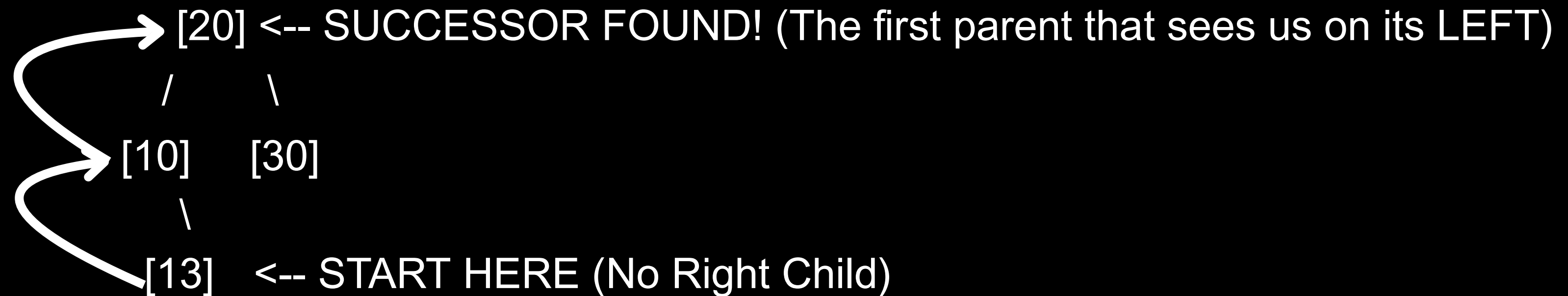
Start at [13]: You look to the right—it's empty. You must climb up to the parent

Move to Parent [10]: You look back down and realize you were the Right Child of 10. This means 10 is smaller than you ($10 < 13$)

10 cannot be the successor. Keep climbing

Move to Parent [20]: You look back down and realize the branch you came from (the 10-13 branch) is the Left Child of 20

20 is the successor



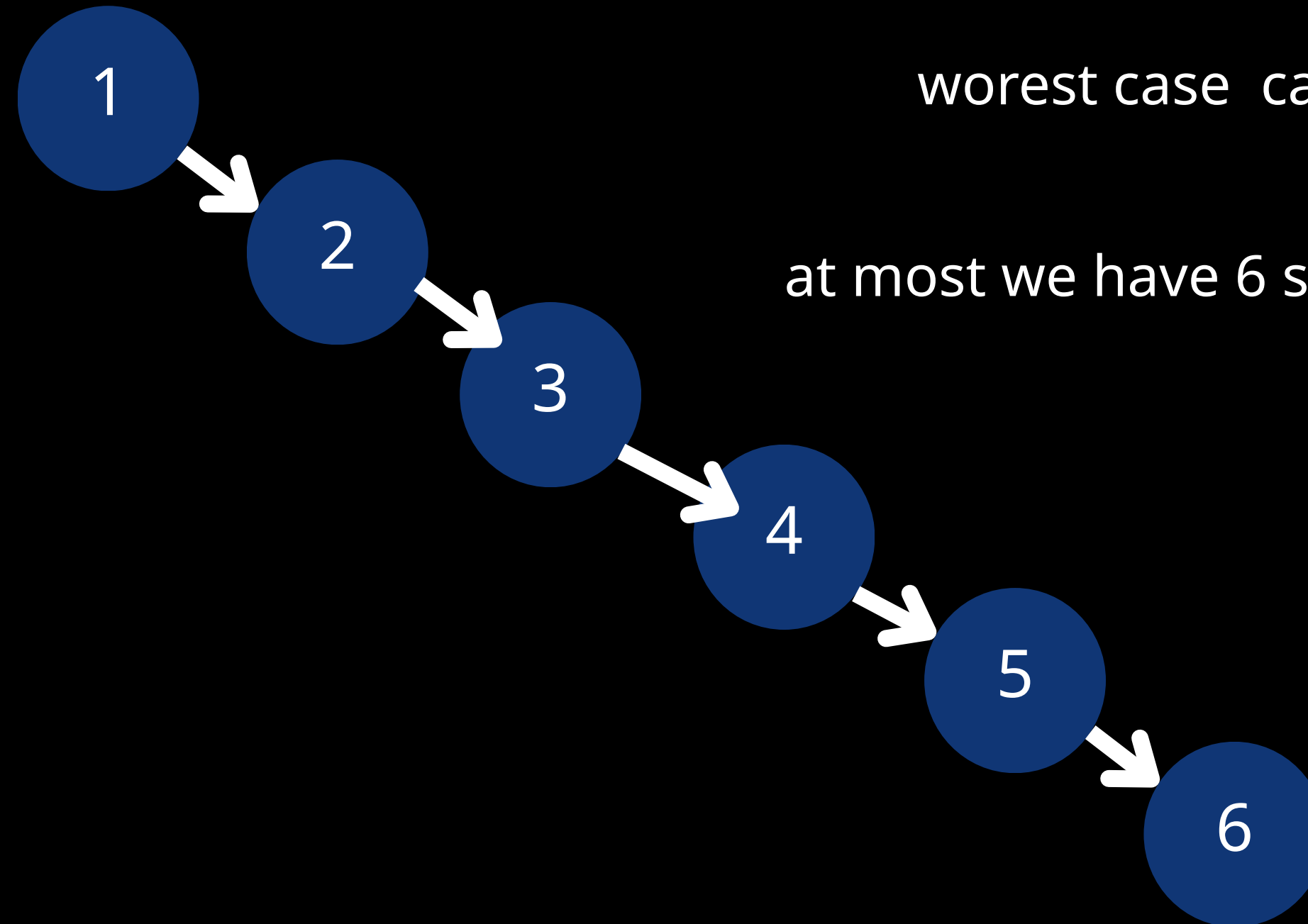
BALANCED TREES

What is a Balanced Tree

A tree is considered balanced if its height is kept small relative to the number of nodes it contains. Specifically, in a balanced tree, we aim for a height of $(\log n)$

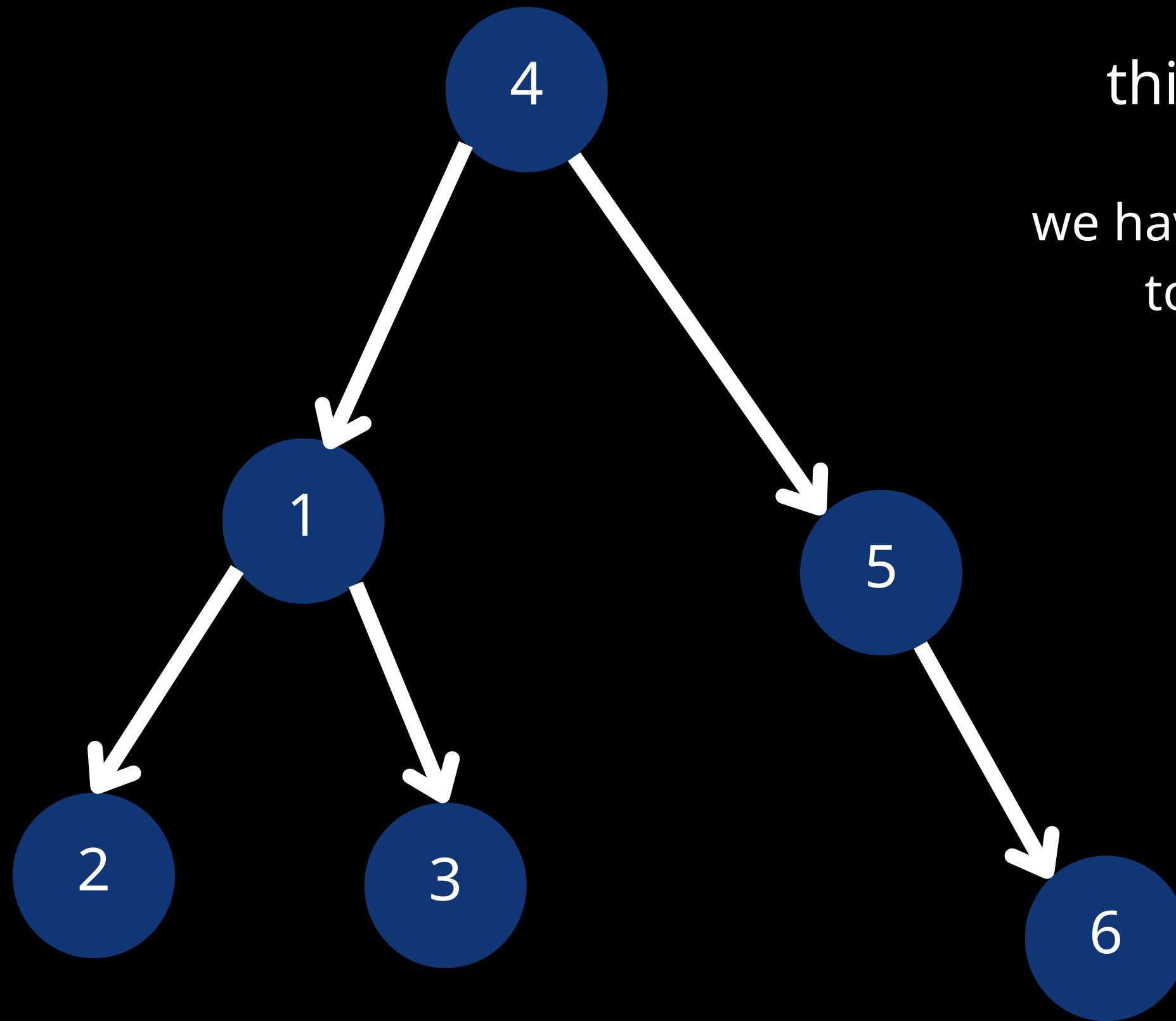
Think of it like a scale. If one side of the scale has ten weights and the other side has zero, the scale is "unbalanced." In a tree, if the left path is 10 nodes deep and the right path is only 1 node deep, searching becomes very slow

lets say we have an a tree of six nodes



worest case cant be $\log n$!!

at most we have 6 steps to reach number 6



this tree are balnced

we have like at most 3 steps
to reach number 6

How to Check if a Tree is Balanced

To check balance, we use the Height of the subtrees. There are two main ways to define "balanced":

- The AVL Rule (Height-Balanced)
- The Calculation Logic

AVL

its Self-Balancing Binary Search Tree

where the difference between heights of left and right subtrees for any node cannot be more than one

we need in avl tree a BF (balance factor)

how to calc the balance factor : not the height of the hoal tree just the subtree

Balance Factor = left subtree height - right subtree height

For every node, the difference in height between the left and right sides can only be -1, 0, or 1

If the difference becomes 2 or -2, the tree is "unbalanced" and immediately fixes itself

How it Fixes Itself : (Using Rotations)

When the tree becomes unbalanced, it performs a Rotation. This is just a fancy way of saying it moves the nodes around to make the tree shorter

rotation types

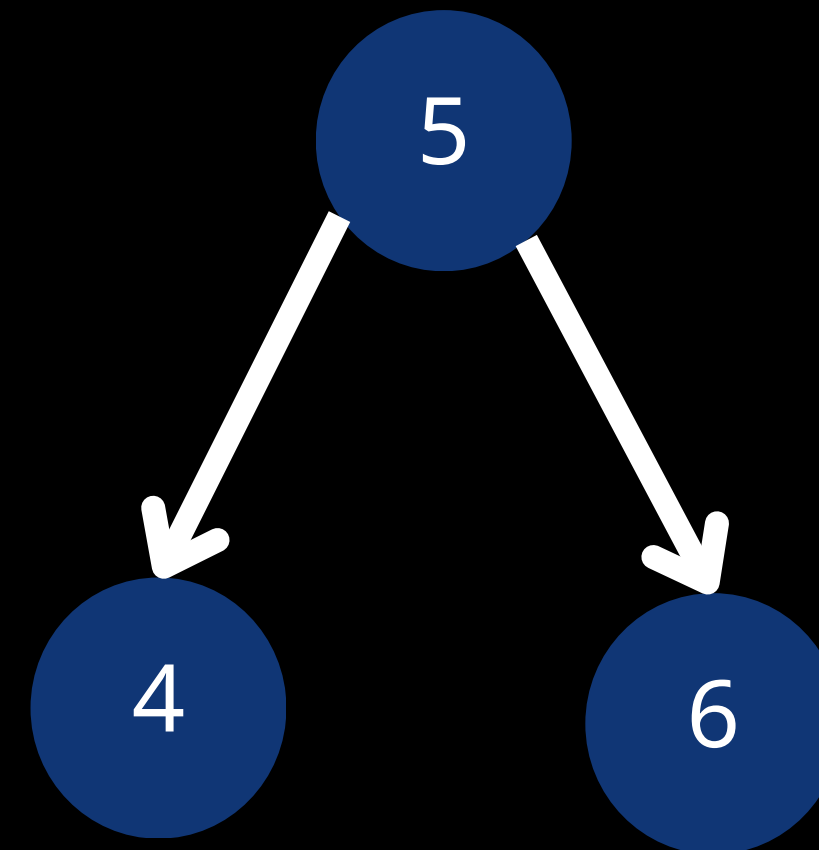
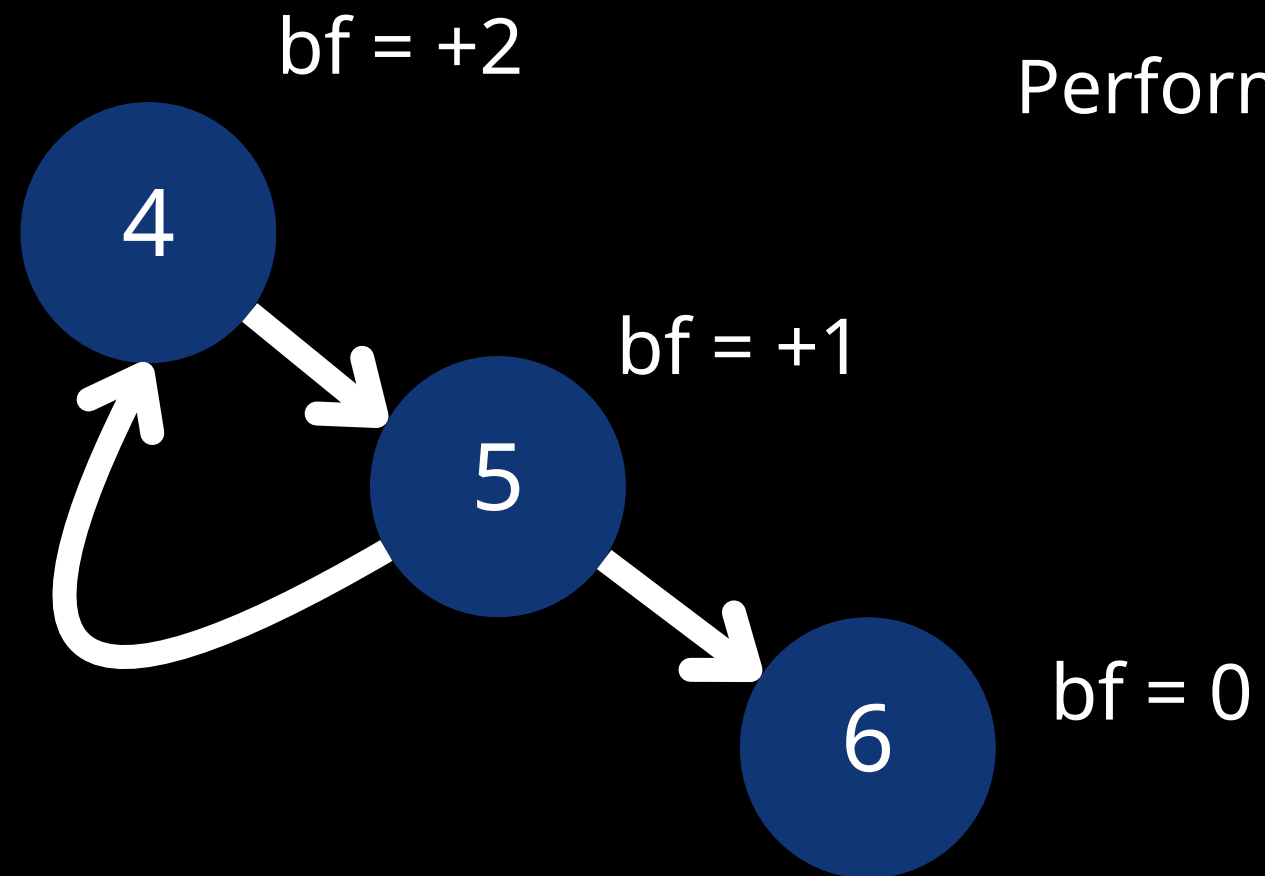
Single Rotation: Used when the tree is a straight line (Left-Left or Right-Right)

Double Rotation: Used when the tree is a "Zig-Zag" (Left-Right or Right-Left)

Left-Left Case:

Occurs when a node is inserted into the left subtree of the left child, causing the balance factor to become more than +1

Perform a single right rotation to fix it



Deff between LL and RR

In LL, the parent's left child becomes the new root, and its right child is handed over

In RR, the parent's right child becomes the new root, and its left child is handed over

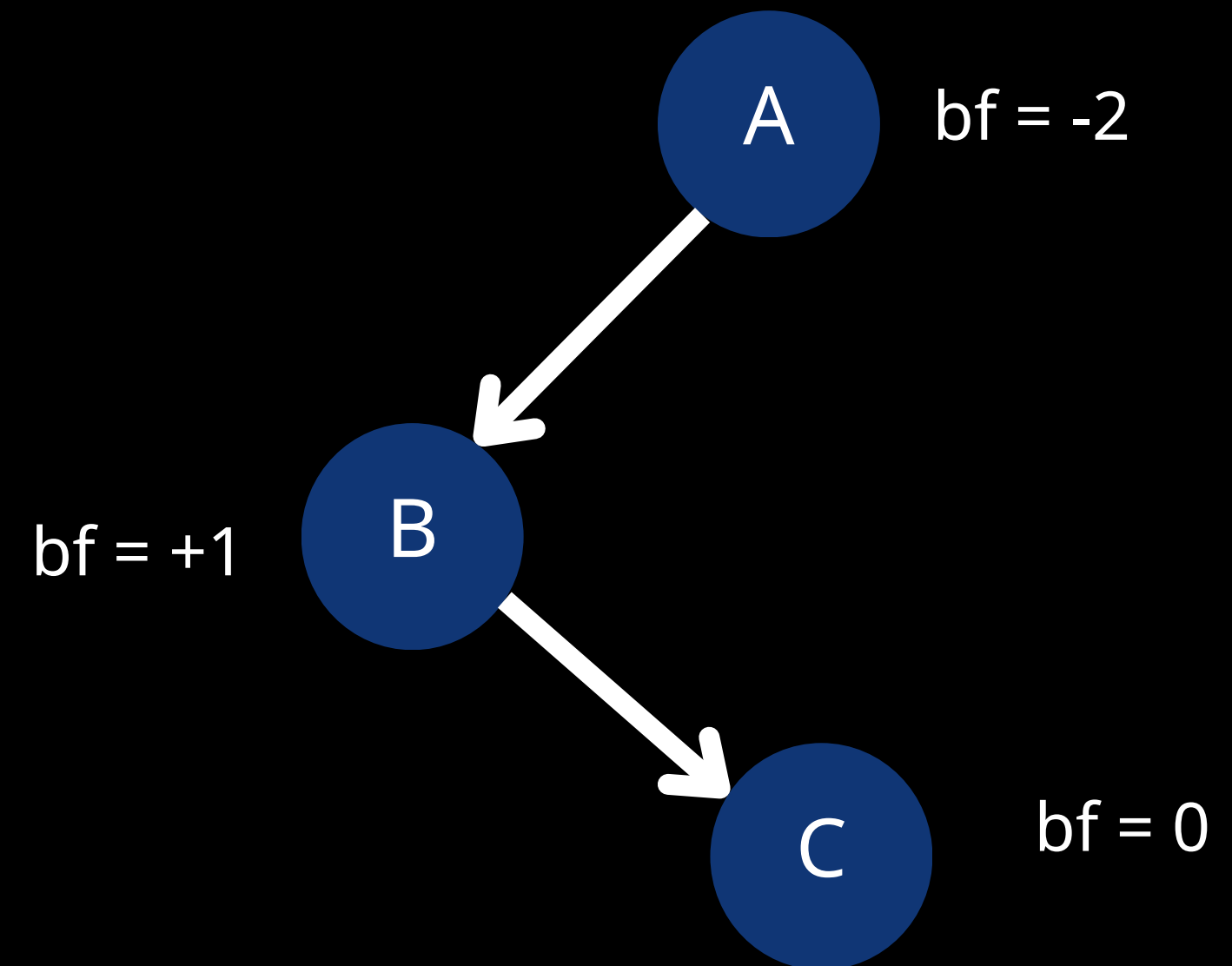
The Left-Right (LR) Case

The Left-Right case is when the unbalanced node is left heavy, but its left child node is right heavy

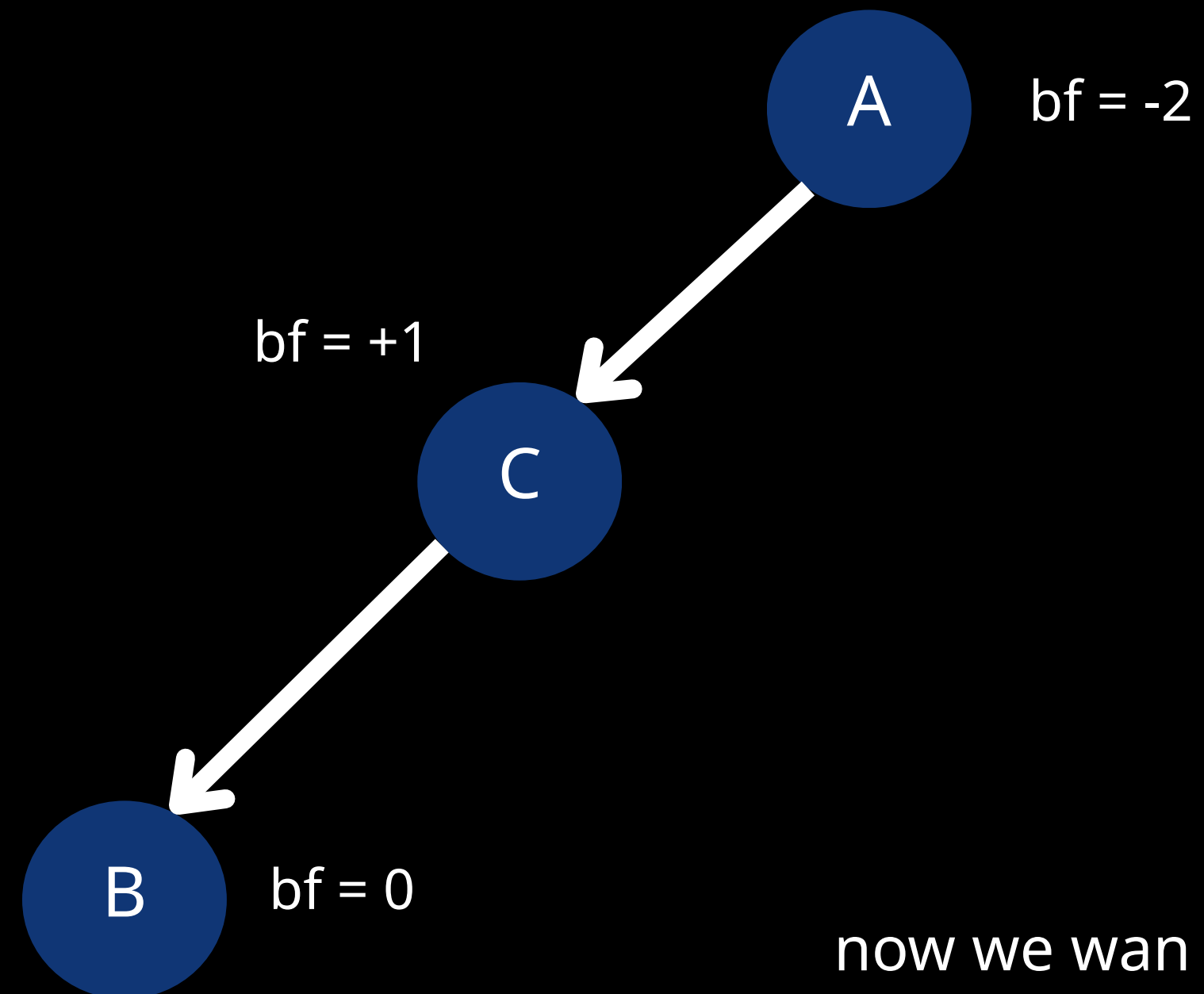
In this LR case, a left rotation is first done on the left child node, and then a right rotation is done on the original unbalanced node

in this tree
how to make it balanced

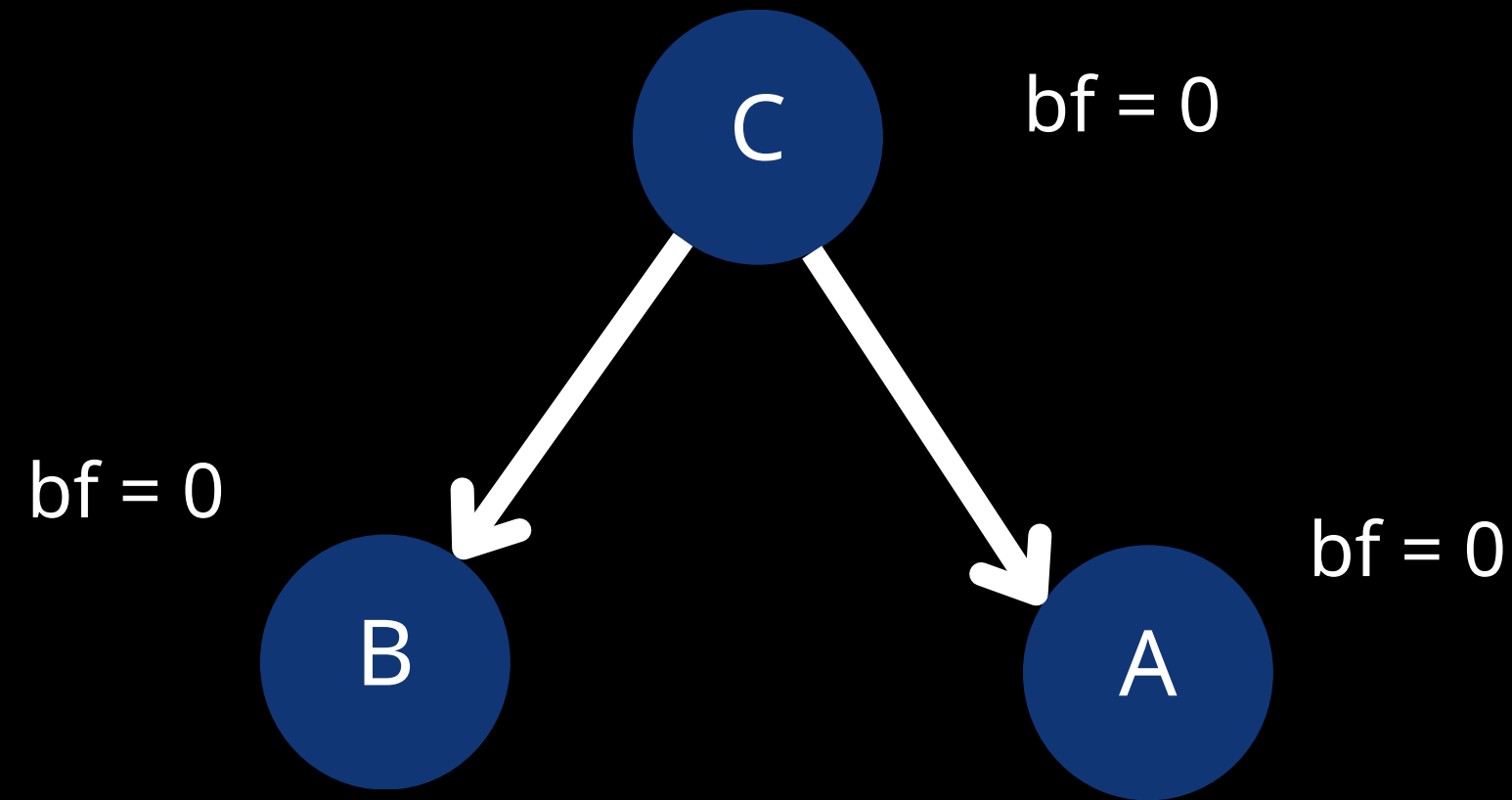
note : B in the Left of A and C in the Right of B



if we perform the RR cuz B in the Left of A this is the tree will be



now we want to make RR to fix the LL
after LL we perform

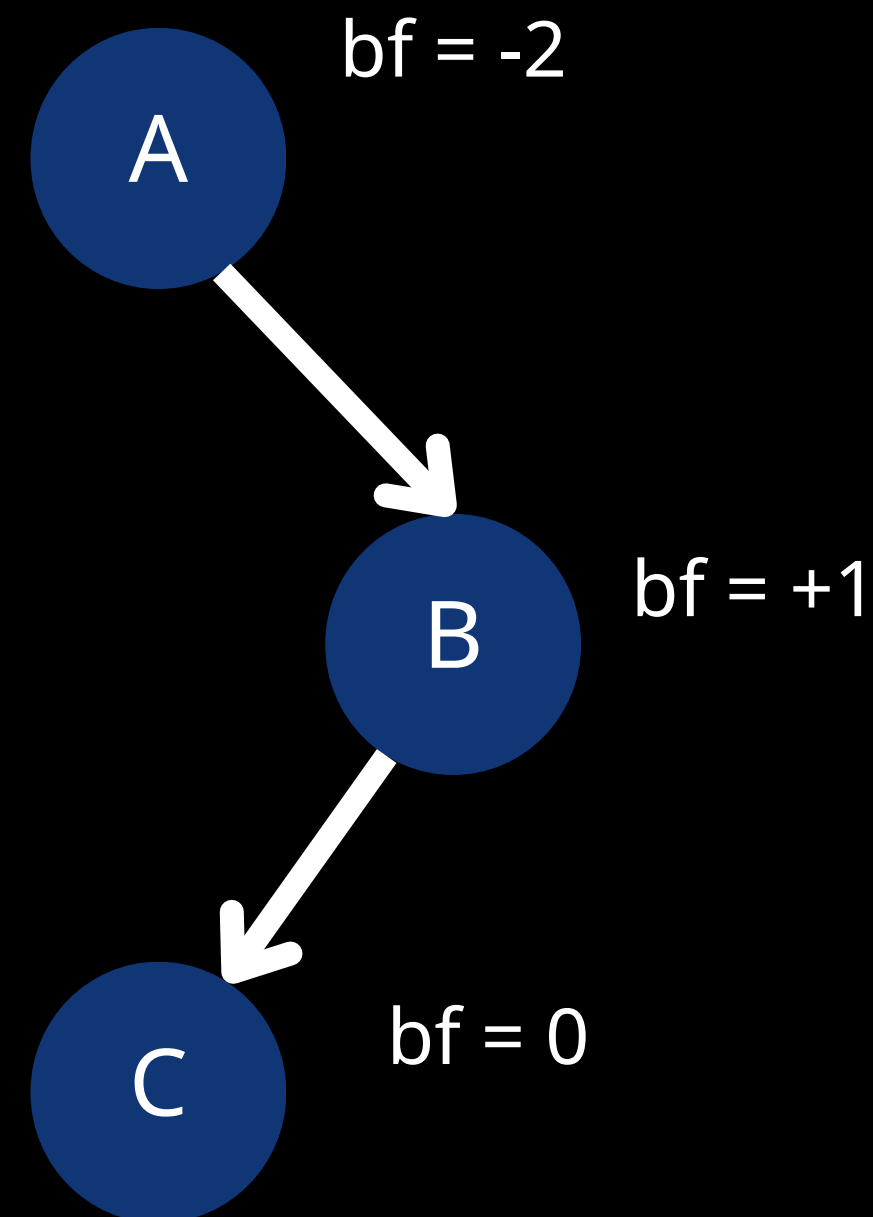


When C is inserted, node A gets unbalanced with a balance factor of -2, so it is left heavy, and its left child B is right heavy, so this is a Left-Right case.

The Right-Left (RL) Case

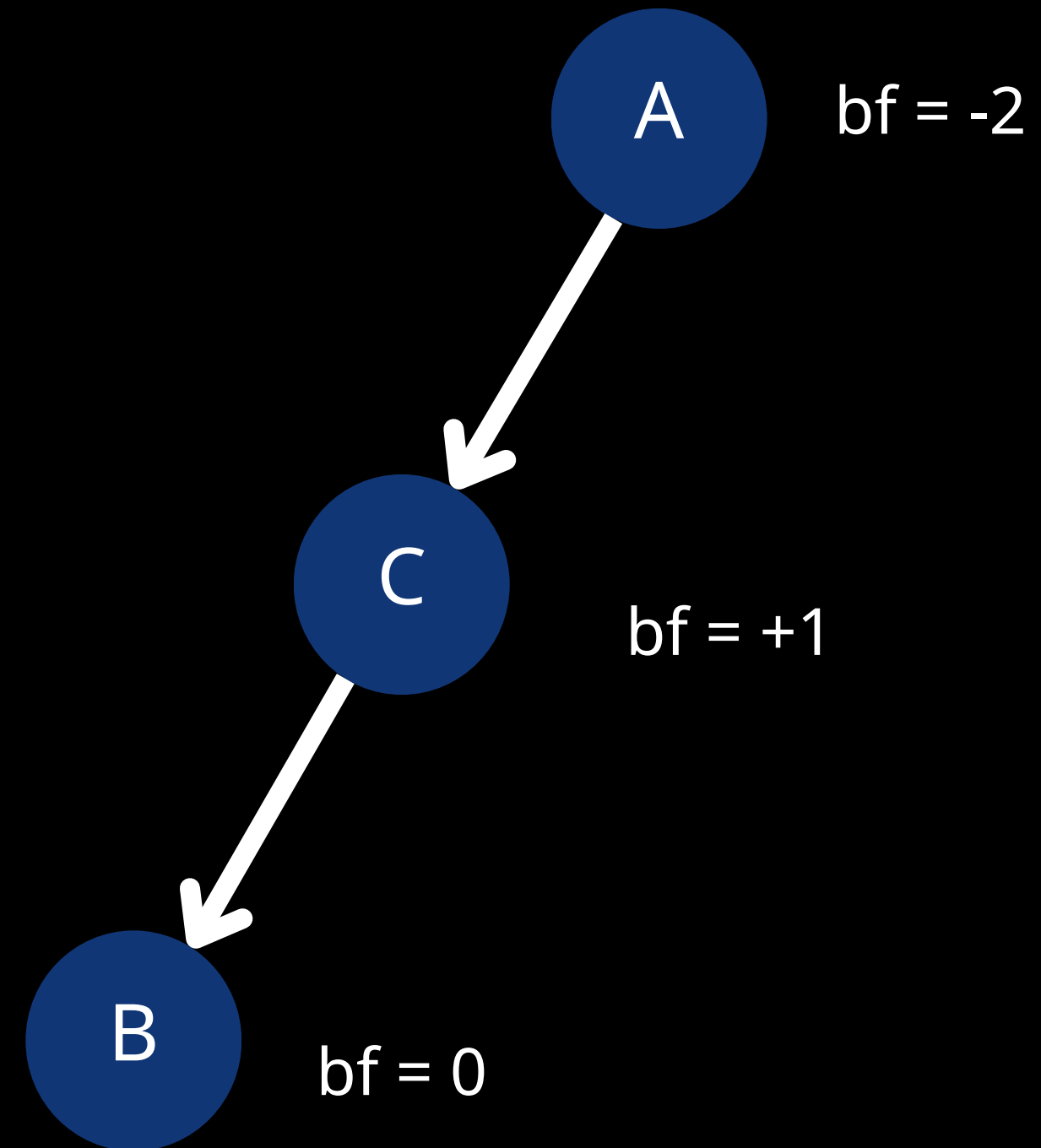
The Right-Left case is when the unbalanced node is right heavy, and its right child node is left heavy

In this case we first do a right rotation on the unbalanced node's right child, and then we do a left rotation on the unbalanced node itself

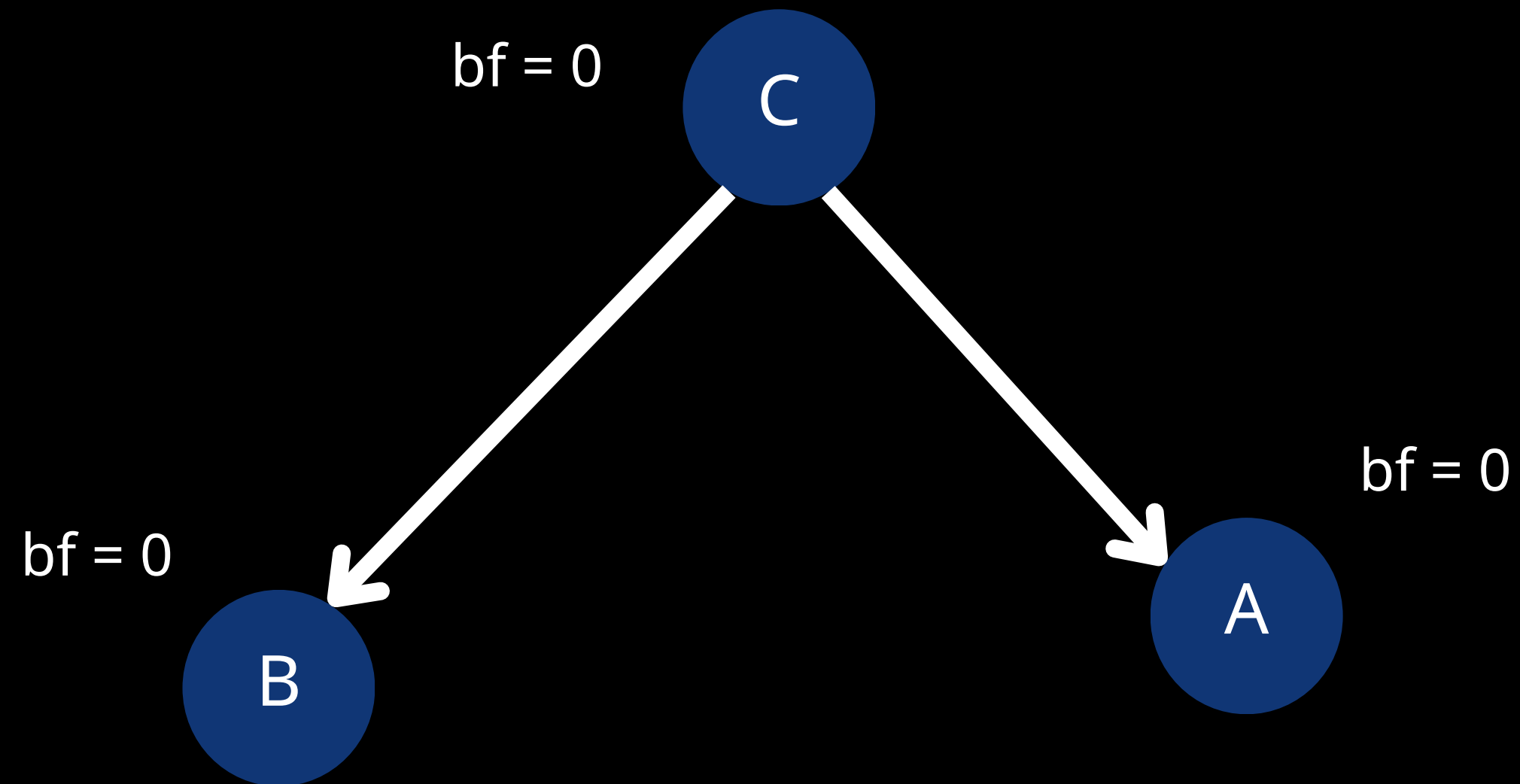


In the first step we perform LL cuz its RR problem

now we have an LL problem



now we perform RR cuz its an LL problem
and thats is we make the tree balanced



Thanks

All code and examples can be found on GitHub <https://github.com/ESSAMMOHAMED1>